

Linear Systems

Design and Analysis of Algorithms

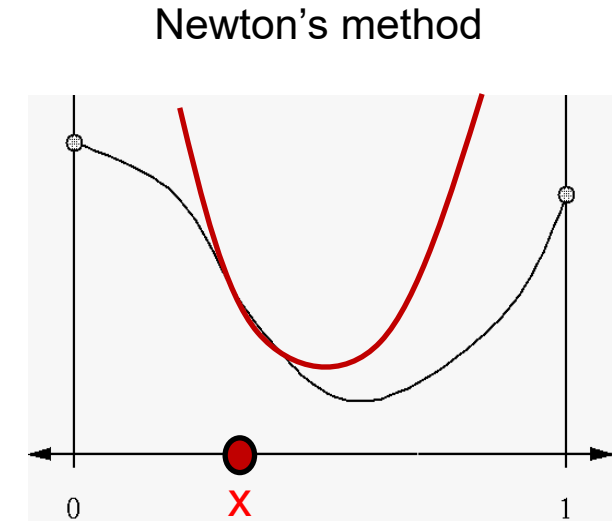
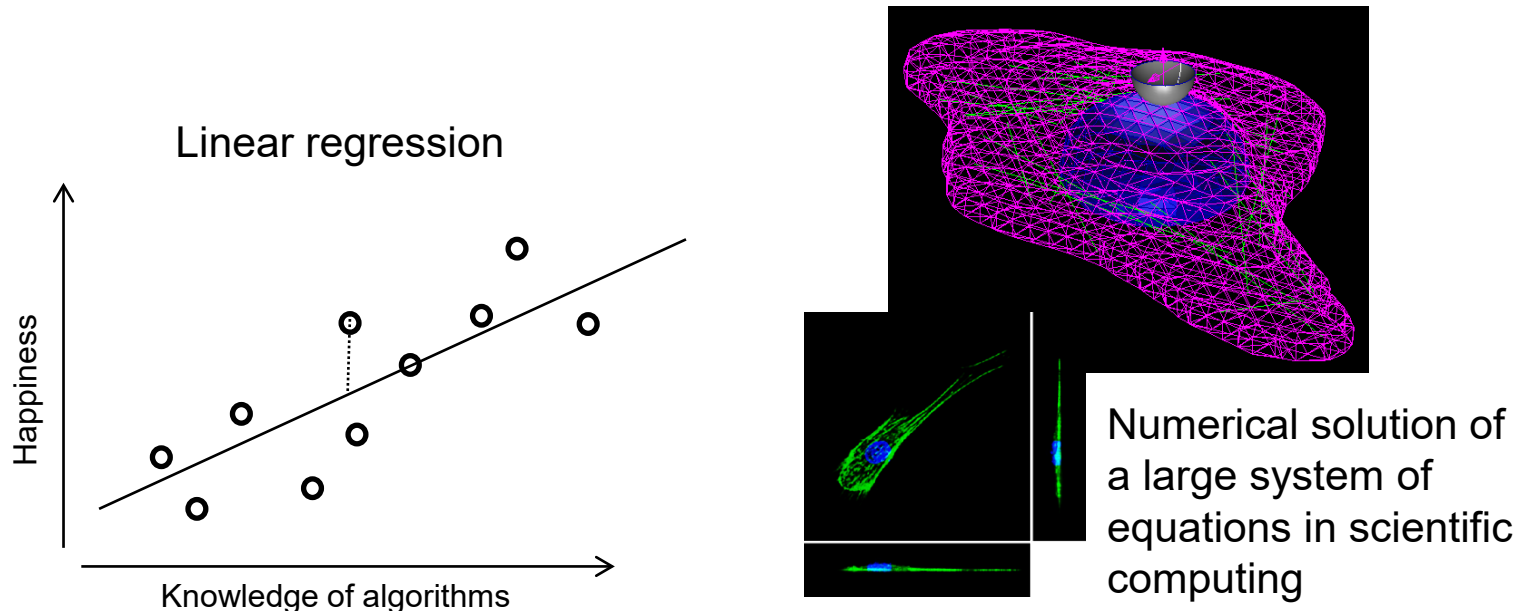
Brian C. Dean

**School of Computing
Clemson University**



Linear Systems are Everywhere

- Least squares regression or solving a system of equations usually involves minimizing a quadratic function giving squared error.
- To minimize a quadratic, set its gradient to zero – this gives a linear system.



Linear Systems

- n unknown variables, n equations

- Example: $x + y = 3$

$$2x - y = 3$$

- Example: $x_1 + 2x_2 + 3x_3 = 10$

$$4x_1 + 5x_2 + 6x_3 = 11$$

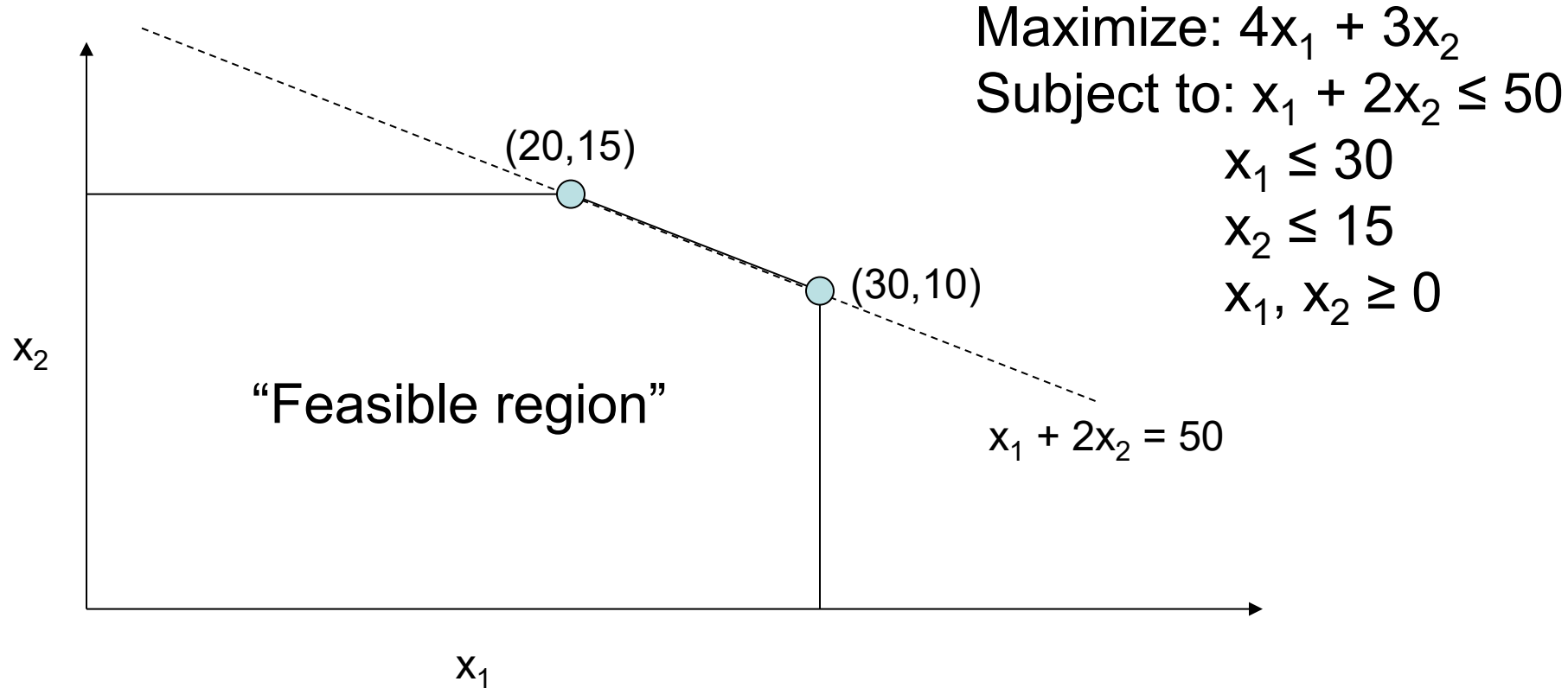
$$7x_1 + 8x_2 + 9x_3 = 12$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 10 \\ 11 \\ 12 \end{bmatrix}$$

$A \qquad x \qquad b$

- What if the system isn't square?
 - More equations than variables? Likely no solution; we often look for a least squares solution minimizing $\|Ax - b\|^2 \rightarrow$ solve square system $A^T A x = A^T b$.
 - More variables than equations? Likely vector space of solutions; we often find the “best” one by minimizing some objective $f(x)$ subject to $Ax = b$.

Linear Programming



The optimal solution will always be a corner point of the feasible region.

Solving Linear Systems with Gaussian Elimination

- Consider solving an $n \times n$ linear system $Ax = b$.
- Example:
$$\begin{array}{rcl} x_1 + 2x_2 + 3x_3 & = & 10 \\ 4x_1 + 5x_2 + 6x_3 & = & 11 \\ 7x_1 + 8x_2 + 9x_3 & = & 12 \end{array}$$
- Solve first equation for x_1 in terms of other variables: $x_1 = 10 - 2x_2 - 3x_3$.
- Plug this in to eliminate x_1 from equations $2 \dots n$, giving a system of $n - 1$ variables, $n - 1$ equations.
- Eventually, we get a single equation in x_n . Solve it, then work backwards to get values of $x_{n-1} \dots x_1$.
- Total running time: $O(n^3)$

Solving Linear Systems

- The “standard” method for solving a linear system via Gaussian elimination takes $O(n^3)$ time.
- With a bit of cleverness, we can solve a linear system in the same amount of time it takes to multiply two $n \times n$ matrices (also $O(n^3)$ time using the most straightforward approach, but slightly faster methods available – e.g., Strassen’s divide and conquer algorithm).
- We can solve $Ax = b$ by inverting A , giving $x = A^{-1}b$; note that matrix inversion also has equivalent complexity to matrix multiplication.
- Unfortunately, this does not scale well enough for very large and sparse systems (Gaussian elimination usually destroys sparsity). For those, we often turn to *iterative refinement*.

Sparse Linear Systems

- Similar meaning to sparse graphs (in fact, we often think of a graph $G(A)$ associated with matrix A , with edge weight A_{ij} on edge (i, j)).
- Typically stored as a list or hash table containing only the m nonzero entries, using $O(m)$ space.
- Matrix-vector multiplication straightforward using either compressed sparse representation, taking only $O(n + m)$ time; many iterative methods based on this.

$$\begin{bmatrix} 1 & 0 & 3 \\ 4 & 5 & 0 \\ 0 & 0 & 9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 10 \\ 14 \\ 27 \end{bmatrix}$$

$A \qquad x \qquad b$

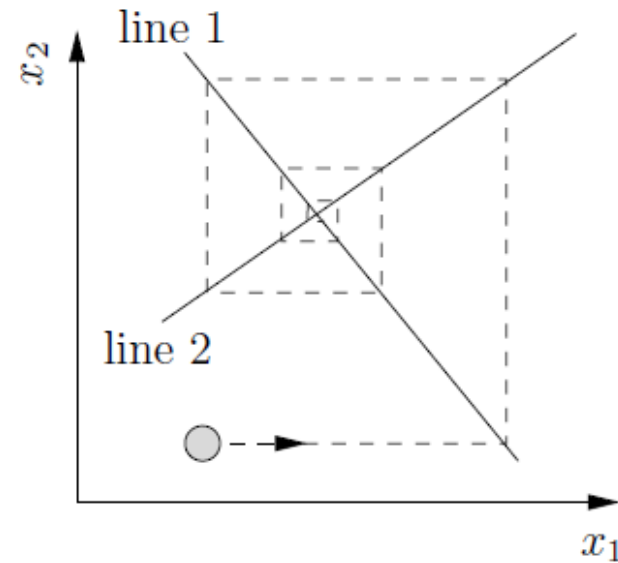
Richardson Iteration

- $Ax = b$ solved by $x = A^{-1}b$.
- Motivated by approximation $A^{-1} \approx I + (I - A) + (I - A)^2 + (I - A)^3 + \dots$
- In each iteration, $x' = x + (b - Ax) = b + (I - A)x$
- Each iteration just matrix-vector multiplication, so $O(n + m)$ time.
- If A positive definite, we get an ε reduction in error (either $\|x - x^*\|$ or $\|x - x^*\|^2$ or $\|b - Ax\|$ or $\|b - Ax\|^2$) after $O(\kappa \log(1/\varepsilon))$ steps.
- Here κ = ratio of largest to smallest eigenvalues of A is a common way to define A 's “condition number”.
- Interpretation as gradient descent...

$$\begin{array}{ccc} \begin{bmatrix} 1 & 0 & 3 \\ 4 & 5 & 0 \\ 0 & 0 & 9 \end{bmatrix} & \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} & = & \begin{bmatrix} 10 \\ 14 \\ 27 \end{bmatrix} \\ & A & x & b \end{array}$$

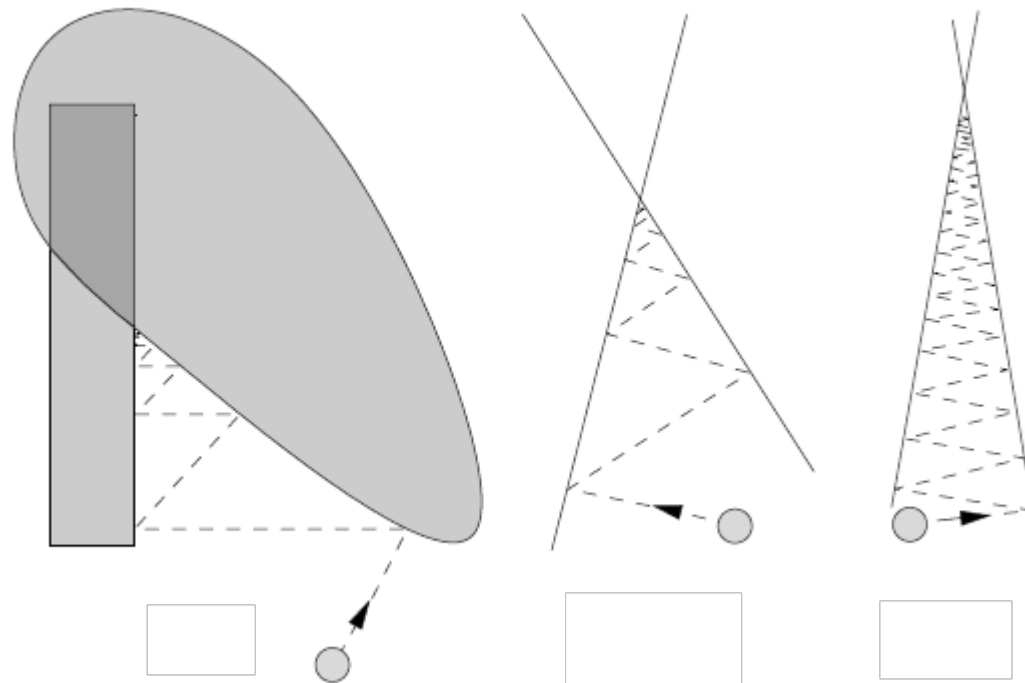
Gauss-Seidel Relaxation (Coordinate Descent)

- Consider solving an $N \times N$ linear system $Ax = b$.
- Example:
$$\begin{aligned}x_1 + 2x_2 + 3x_3 &= 10 \\4x_1 + 5x_2 + 6x_3 &= 11 \\7x_1 + 8x_2 + 9x_3 &= 12\end{aligned}$$
- Using iterative refinement:
 - Guess an initial solution $x = (x_1 \dots x_N)$
 - Solve equation 1 for x_1 to update x_1 .
 - Solve equation 2 for x_2 to update x_2 .
 - Etc... (repeating cyclically)
- $O(n + m)$ for one complete pass to update all variables.
- Positive definiteness of A one condition guaranteeing convergence.



Successive Projection (Kaczmarz's Method)

- One way to find a point in the intersection of several convex sets (also convex) is by cyclically projecting onto each one.



An Optimization Outlook

- If A positive definite, solving $Ax = b$ equivalent to minimizing $f(x) = \frac{1}{2} x^T Ax - b^T x$ (strongly convex).
 - Observe that $\nabla f(x) = Ax - b$, so $\nabla f(x) = 0$ means $Ax = b$.
 - Compare to solving $ax = b$ versus minimizing $f(x) = \frac{1}{2} ax^2 - bx$.
- Recall Richardson iteration in this case took $O(\kappa \log(1/\epsilon))$ steps (each running in linear time) to achieve an ϵ reduction in error; this is because it's equivalent to gradient descent on f .
 - Note condition number κ means the same thing equivalently as A 's condition number of the condition number of f .
- A popular approach here (equivalent to accelerated gradient descent is the *conjugate gradient method*), requiring only $O(\sqrt{\kappa} \log(1/\epsilon))$ linear-time steps.